

# Trajectory Planning for UAV-Assisted Data Collection in IoT Network: A Double Deep Q Network Approach

Final paper presentation of Learning for Autonomous Systems course – UniTs

June 2026



Tavano Matteo

SM380057

## Reference

**This presentation is based on the following paper:**

Wang, S., Qi, N., Jiang, H., Xiao, M., Liu, H., Jia, L., & Zhao, D. (2024).  
*Trajectory Planning for UAV-Assisted Data Collection in IoT Network: A Double  
Deep Q Network Approach.*  
**Electronics**, 13(8), 1592.  
<https://doi.org/10.3390/electronics13081592>

# Problem Introduction & Motivation

**Settings:** UAV-assisted IoT data collection in a 3D urban environment with obstacles and wireless jammers, modeled as **Markov Decision Process (MDP)**.

**Challenge:** optimize the UAV 3D trajectory in order to **maximize data collected** from ground IoT devices, while **avoiding obstacles** and **jammers** interference, and operating within a limited battery budget.

**Approach:** the authors propose a **Double Deep Q-Network (DDQN)** based agent that learns an autonomous, energy-efficient flight strategy through **deep reinforcement learning**.

# System Model: Overview

The scenario consists of **U stationary IoT devices** and **J ground jammers** distributed across a 2D plane.

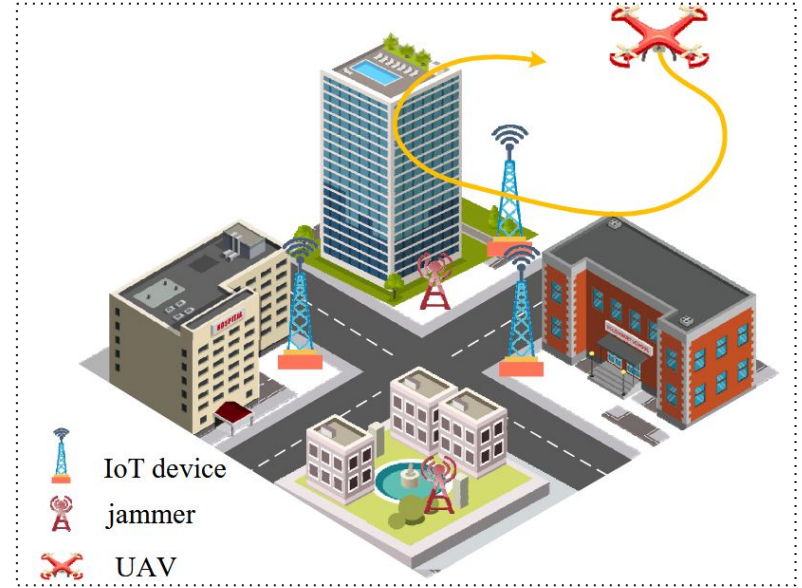
The UAV operates over a mission of duration **T**, discretized into **N** equal time steps, to maximize data collection.

Variables at time step  $n$ :

$$\mathbf{q}_n = [x_n, y_n, h_n] \in \mathbb{R}^3, \forall n \in N$$

$$\mathbf{q}_n^u = [x_n^u, y_n^u, 0] \in \mathbb{R}^3, \forall n \in N$$

$$\mathbf{q}_n^j = [x_n^j, y_n^j, 0] \in \mathbb{R}^3, \forall n \in N$$



**Action space and feasible actions:**

$$a_x, a_y, a_z \in \{-1, 0, 1\} \quad \mathbf{a}_n = [a_x, a_y, a_z] \in \mathcal{A}, \forall n \in [1, N]$$

Evolution of the UAV position:  $\mathbf{q}_{n+1} = \mathbf{q}_n + \mathbf{a}_n$

Environment includes a set of **B obstacles**.

# System Model: Channel & SINR

Channel Gain:

$$g_n^u = \begin{cases} \beta_{\text{LoS}} + \alpha_{\text{LoS}} \log_{10}(d_n^u) + \eta_{\text{LoS}}(\text{dB}), & \text{if LoS} \\ \beta_{\text{NLoS}} + \alpha_{\text{NLoS}} \log_{10}(d_n^u) + \eta_{\text{NLoS}}(\text{dB}), & \text{otherwise} \end{cases}, \forall n \in [1, N]$$

Interference received:

$$I_n = \sum_{j=1}^J P_j 10^{\frac{g_n^j}{10}}$$

Signal-to-interference plus noise ratio:

$$\text{SINR}_n^u = \frac{P_u 10^{\frac{g_n^u}{10}}}{I_n + \sigma^2}, \forall n \in [1, N]$$

Throughput:

$$R_n^u = B \log_2(1 + \text{SINR}_n^u)$$

**Key Assumption - TDMA access:**

At each communication time step, the UAV can collect data from only one ground device, and only the device with remaining data and the highest **SINR** can establish a communication link with the UAV at the current time step  $n$ .

# Problem Formulation

The algorithm aims to optimize the trajectory of the UAV in order to **maximize the amount of data collected** from the ground equipment during the mission time **T**.

$$\max_{\mathbf{a}_n} \sum_{n=1}^T \sum_{u=1}^U l_n^u R_n^u \delta_n$$

$$s.t. \mathbf{q}_n \notin \mathcal{B}, \forall n \in [1, N]$$

$$b_n > 0, \forall n \in [1, N]$$

$$SINR_n^u \geq \gamma_{th}, \forall u \in \mathcal{U}, n \in [1, N]$$

**Channel Constraint:**

$$\sum_{u=1}^U l_n^u \leq 1, \forall u \in \mathcal{U}, n \in [1, N]$$

**Link status:**  $l_n^u \in \{0, 1\}$

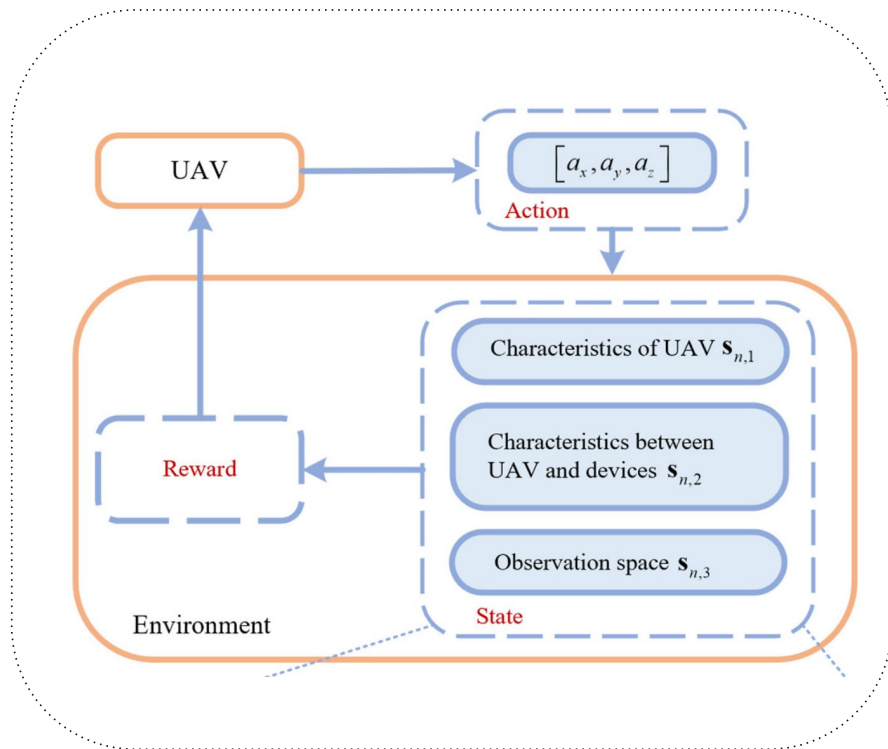
# Problem Formulation: Challenges

This optimization problem is challenging, due to its **non-convexity** and **unknown environment**. Consequently, conventional model-based approaches are inapplicable.

To address these challenges, the problem is modeled as a **Markov Decision Process (MDP)**, where the UAV sequentially interacts with the environment while making trajectory decisions.

A **Reinforcement Learning (RL)** approach is adopted to enable the UAV to learn an effective flight policy through trial-and-error interactions, without requiring prior knowledge of the environment.

# MDP Formulation



State Space  $S$ :  $\mathbf{s}_n = (\mathbf{s}_{n,1}, \mathbf{s}_{n,2}, \mathbf{s}_{n,3})$

$$\mathbf{s}_{n,1} = \{\mathbf{q}_n, b_n, L_n\}$$

$$\mathbf{s}_{n,2} = \{\mathbf{q}_n^u, l_n^u, \text{SNR}_n^u, d_n^u, D_n^u\}$$

$$\mathbf{s}_{n,3} = \{o_n, o_{n+1}\}$$

Action Space  $A$ :  $\mathbf{a}_n = [a_x, a_y, a_z] \in \mathcal{A}, \forall n \in [1, N]$

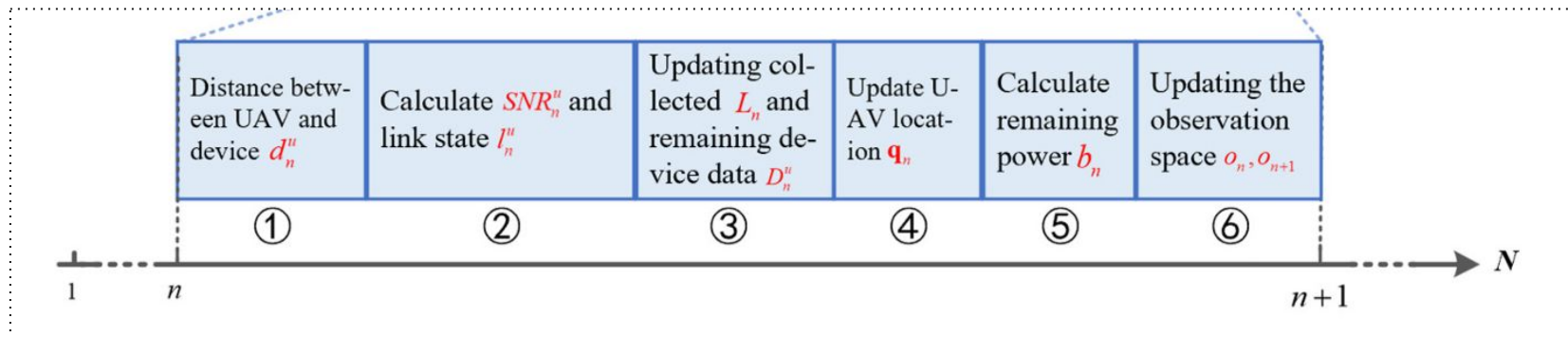
Reward  $R$ :  $\mathbf{r}_n = r_{n,1} - r_{n,2} - r_{n,3}$

Throughput reward:  $r_{n,1} = \sum_{u=1}^U l_n^u R_n^u \delta_n$

Transition Probability  $P$ .



# Pipeline of Data Collection

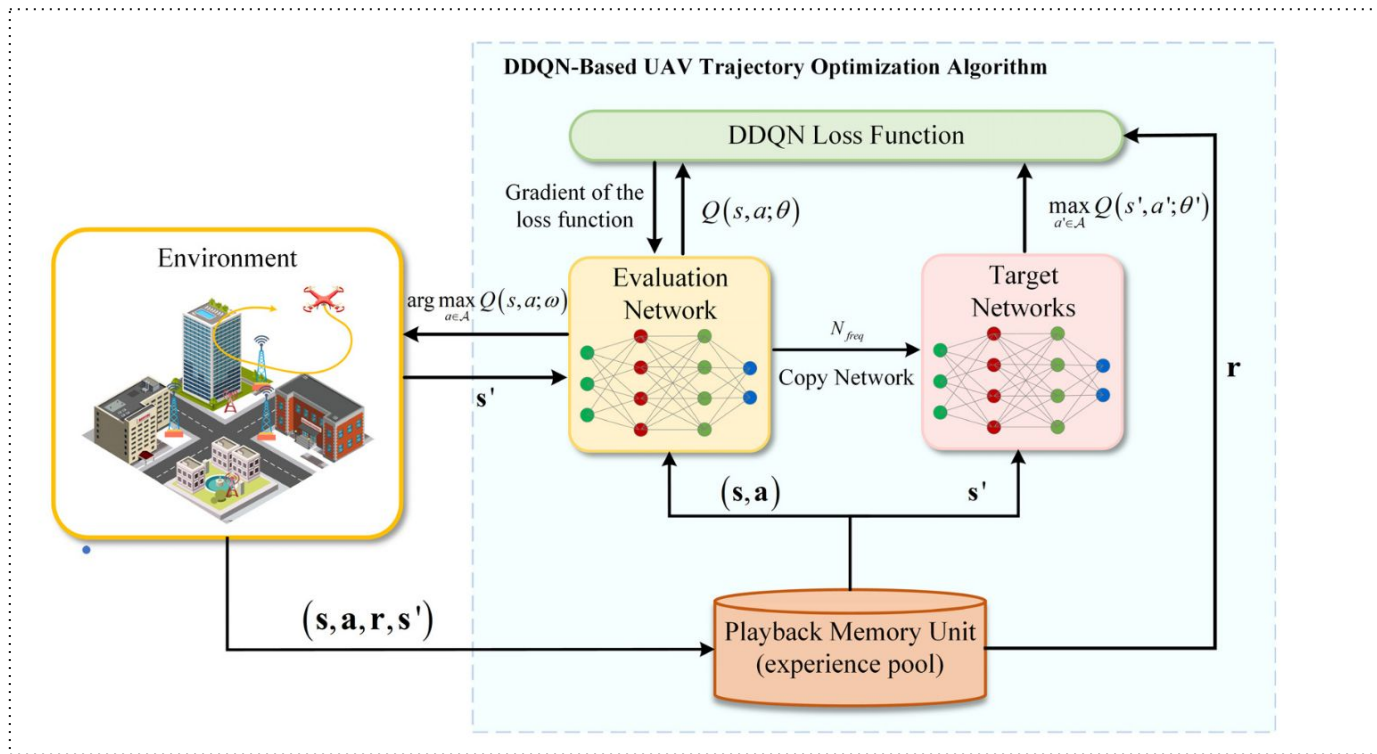


# DDQN Algorithm

The DDQN builds on the standard DQN framework, which approximates the **action-value** function using a neural network trained via deep Reinforcement Learning.

Both DQN and DDQN maintain two networks, an **online network  $\theta$**  and a **target network  $\theta'$** , but they differ in how the **TD target** is computed:

- **DQN:**  $\theta'$  both selects and evaluates the action (**overestimation bias**).
- **DDQN:**  $\theta$  selects the best action,  $\theta'$  evaluates (**decoupled, reduced bias**).



---

**Algorithm 1:** DDQN-Based UAV Trajectory Optimization Algorithm

---

Initialize replay memory  $\mathcal{D}$ , the online network parameters  $\theta$ , the target network parameters  $\theta' = \theta$ , and the target network update period  $N_{freq}$ .

- 1: **for** episode = 0, 1, ..., M - 1 **do**
  - 2: Randomly generating the location of the IoT devices  $\mathbf{q}_0^u$ , the location of the UAV  $\mathbf{q}_0$ , the location of the jammers  $\mathbf{q}_0^j$ , transmission power of UAV  $P$ , and interference power  $P_j$ .
  - 3: Time step  $n = 0$ , initialization of the environment and state  $\mathbf{s}_n$  of the UAV
  - 4: **while**  $b_n \geq 0$  **do**
  - 5: choose action  $\mathbf{a}_n$  with  $\epsilon$ -greedy policy, i.e.,
  - 6: 
$$\mathbf{a}_n = \begin{cases} \text{randomly select from } \mathcal{A} & w.p. \epsilon \\ \arg\max_{\mathbf{a} \in \mathcal{A}} Q(\mathbf{s}_n, \mathbf{a}_n) & w.p. 1-\epsilon \end{cases}$$
  - 7: take action  $\mathbf{a}_n$ , state  $\mathbf{s}_n$ , next state  $\mathbf{s}_{n+1}$ , and reward  $\mathbf{r}_n$
  - 8: store  $(\mathbf{s}_n, \mathbf{a}_n, \mathbf{s}_{n+1}, \mathbf{r}_n)$  in replay memory  $\mathcal{D}$
  - 9:  $\mathbf{s}_n \leftarrow \mathbf{s}_{n+1}$
  - 10:  $n = n + 1$
  - 11: **end while**
  - 12: randomly sample a minibatch from  $\mathcal{D}$
  - 13: 
$$y_n = \begin{cases} \mathbf{r}_{n+1}, & \text{if } \mathbf{s}_{n+1} \text{ is terminal} \\ \mathbf{r}_{n+1} + \gamma Q\left(\mathbf{s}_{n+1}, \arg\max_a Q(\mathbf{s}_{n+1}, \mathbf{a}; \theta_t), \theta'_t\right), & \text{otherwise} \end{cases}$$
  - 14: do a gradient descent step with loss  $\|y_n - Q(\mathbf{s}_n, \mathbf{a}_n; \theta)\|^2$
  - 15: Replace target network parameters  $\theta' \leftarrow \theta$  when  $n = N_{freq}$
  - 16: **end for**
-

# Experimental Setup

**Environment:** 50 x 50 x 10 m.

UAV positioned at center, with random attitude.

**Training and Test set:** 80.000 episodes each. Best-saved model checkpoint.

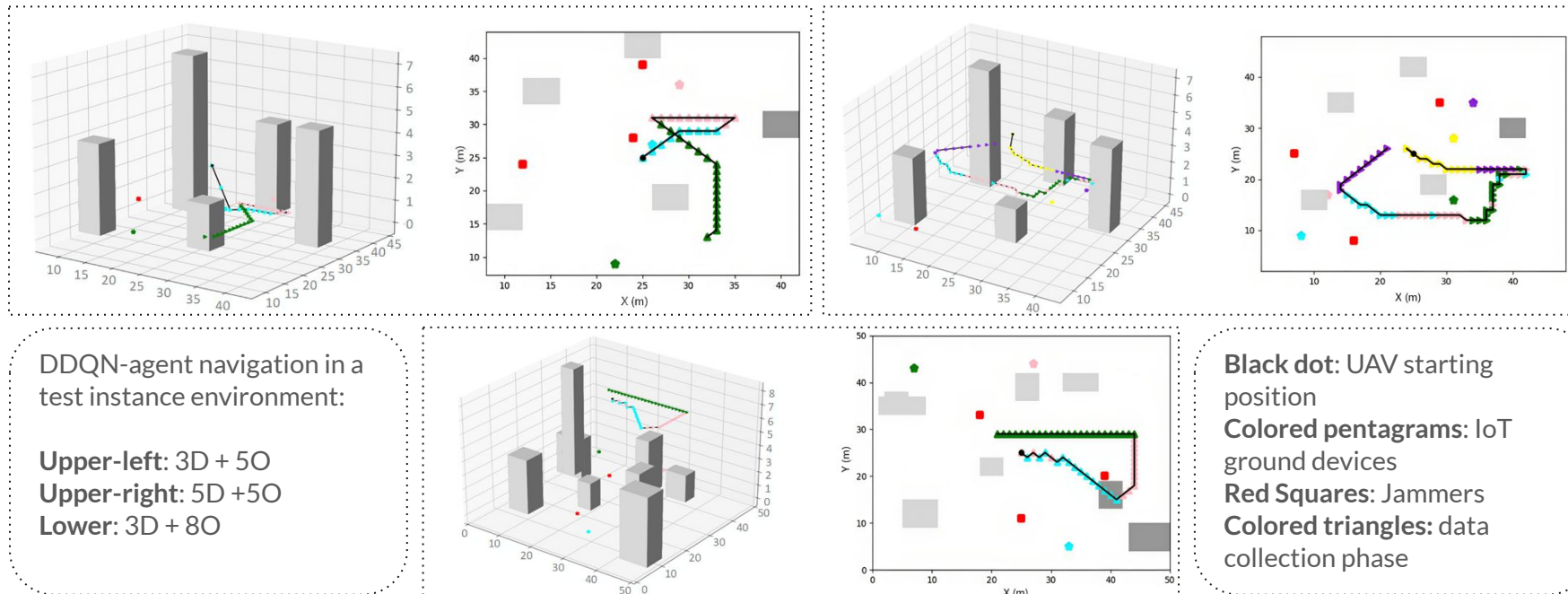
**Algorithms Compared (Baseline):**

- Q-Learning;
- Proximal Policy Optimization (PPO);
- Standard DQN;
- Dueling DQN.

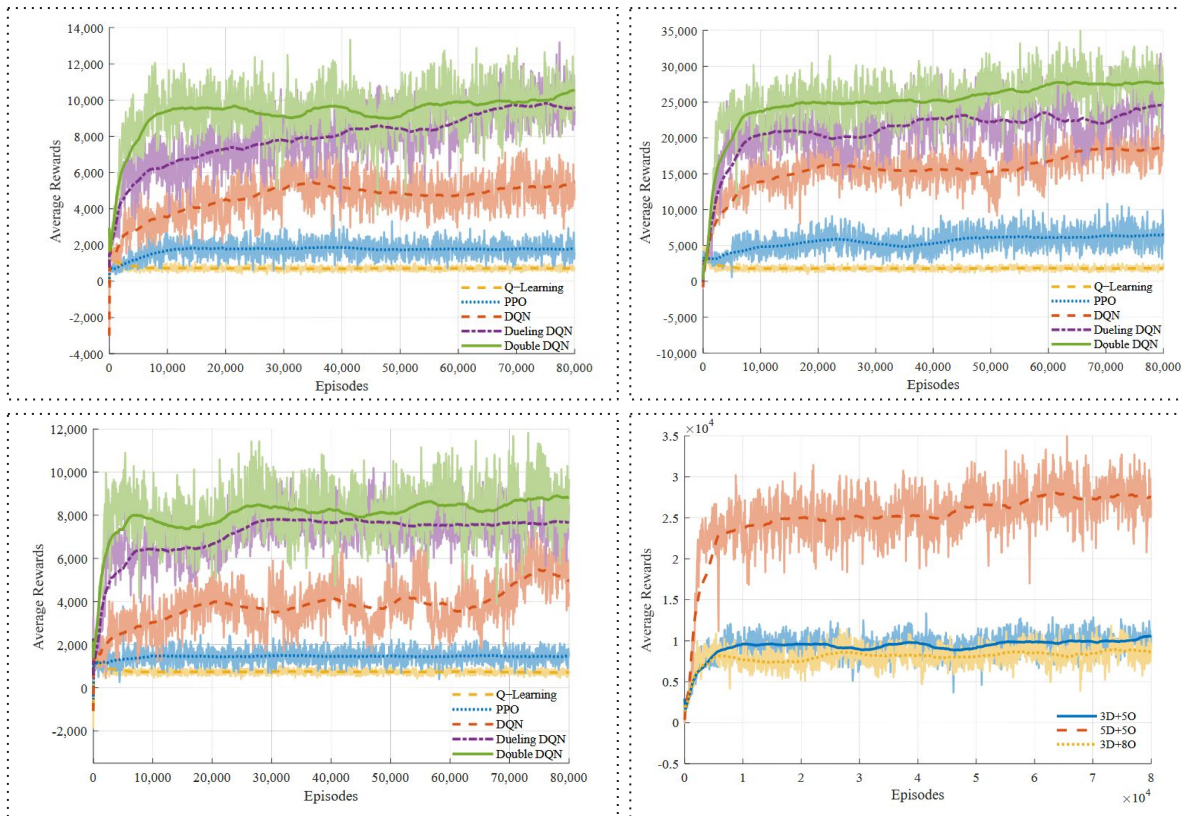
| Parameter                | Value  |
|--------------------------|--------|
| Learning rate $\alpha$   | 0.0004 |
| Discount factor $\gamma$ | 0.99   |
| $\epsilon$               | 0.01   |
| $N_{\text{freq}}$        | 100    |
| Minibatch                | 32     |
| Iterations               | 80000  |

| Scenario | IoT Devices | Jammers | 3D Obstacles |
|----------|-------------|---------|--------------|
| 3D + 5O  | 3           | 3       | 5            |
| 5D + 5O  | 5           | 5       | 5            |
| 3D + 8O  | 3           | 3       | 8            |

# DDQN Data collection - Test scenarios

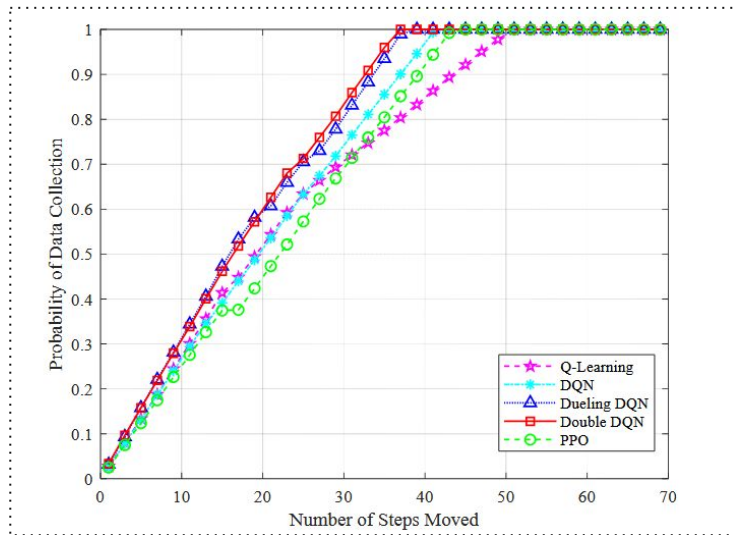


# Learning Curves - Training



Upper-left: 3D+5O  
Upper-right: 5D+5O  
Bottom-left: 3D+8O  
Bottom-right:  
3 scenarios comparison  
(DDQN only)

# Data Collection Efficiency & Training Cost



DDQN reaches **full data collection** in fewer steps than all other algorithms, with training time in line with DQN and Dueling DQN.

| Method      | 3D+5O   | 5D+5O   | 3D+8O   |
|-------------|---------|---------|---------|
| Q-learning  | 1.34460 | 1.28929 | 1.39889 |
| PPO         | 1.03520 | 1.63133 | 1.27002 |
| DQN         | 1.75778 | 4.26592 | 1.80001 |
| Dueling DQN | 2.05745 | 4.35702 | 2.13843 |
| DDQN        | 1.63612 | 3.95510 | 1.71541 |



# Conclusions

## What the paper demonstrates:

Across all tested scenarios, the DDQN-based approach achieved the **highest cumulative reward** compared to all baselines.

The trained agent **navigated** the 3D environment collecting data from IoT devices while avoiding obstacles and jammer interference.

Compared to all baselines, the proposed approach required **fewer steps** to complete **full data collection** and maintained competitive training times.

## Critical Observations:

**Convergence** to the **optimal policy**  $\pi^*$  was never demonstrated, the suboptimality gap remains unknown.

Test performance is assessed on a single fixed instance per scenario, limiting the **generalizability** of the comparison.

Several **implementation** and **hyperparameters details** are **not discussed**, limiting reproducibility.

**Thanks for listening !**